

Machine Learning Roadmap

From Scratch to Job-Ready — A Complete Ordered Learning Path

Six phases, sequenced from programming and math foundations through core ML, deep learning, specialization, and portfolio/interview prep — with a suggested timeline and curated resources.

Prepared by Akshit Sharma | 8510968871 | akshubhardwaj231@gmail.com | akshbuilds.tech

Contents

- Phase 0: Before You Start
- Phase 1: Programming Foundations
- Phase 2: Math Foundations
- Phase 3: Core Machine Learning
- Phase 4: Deep Learning
- Phase 5: Specialize (choose a path)
- Phase 6: Portfolio & Job Prep
- Suggested Timeline
- Core Resource List
- Mistakes to Avoid

Phase 0: Before You Start

- **Goal:** set expectations and pick a track.
- Time to first job-ready milestone: **8–12 months** at 10–15 hrs/week (faster if full-time).
- **ML Engineer / Applied ML** track — build products, APIs, pipelines.
- **Data Scientist / Research** track — heavier math, experimentation, papers.
- This roadmap covers the shared foundation (Phases 1–4), then branches at Phase 5.

Phase 1: Programming Foundations (3–4 weeks)

1.1 Python (mandatory)

- Syntax, data types, loops, functions, OOP basics
- List/dict comprehensions, generators, `*args/**kwargs`
- File I/O, exception handling, virtual environments (venv/conda)
- **Practice:** solve 50 problems on HackerRank/LeetCode (easy tier)

1.2 Core Libraries

Library	Use
NumPy	arrays, vectorized math, broadcasting
Pandas	dataframes, cleaning, groupby, merging
Matplotlib / Seaborn	plotting, EDA visuals

1.3 Tools

- Git & GitHub (branches, commits, PRs)
- Jupyter Notebook / Google Colab
- Basic command line (cd, ls, pip, grep)

Checkpoint project

Load a messy CSV, clean it with Pandas, produce 5 visualizations.

Phase 2: Math Foundations (4–6 weeks, run alongside Phase 1)

You don't need a math degree — you need working intuition and the ability to read a formula and know what it's doing.

2.1 Linear Algebra

- Vectors, matrices, matrix multiplication, transpose
- Dot product, norms, eigenvalues/eigenvectors (intuition, not proofs)
- Why it matters: every ML model is matrix operations under the hood

2.2 Probability & Statistics

- Mean, median, variance, standard deviation
- Probability distributions (Normal, Binomial, Poisson)

- Bayes' theorem, conditional probability
- Hypothesis testing, p-values, confidence intervals
- Correlation vs causation

2.3 Calculus

- Derivatives, partial derivatives, chain rule
- Gradients — this is literally how models learn (gradient descent)
- You do NOT need to solve integrals by hand; focus on why gradients minimize loss

Resources

3Blue1Brown — *Essence of Linear Algebra / Calculus* (YouTube); Khan Academy Statistics.

Checkpoint

Implement gradient descent from scratch in NumPy (no ML library) to fit a line to data.

Phase 3: Core Machine Learning (6–8 weeks)

3.1 Concepts First

- Supervised vs unsupervised vs reinforcement learning
- Train/validation/test split, overfitting vs underfitting
- Bias-variance tradeoff
- Cross-validation
- Evaluation metrics: accuracy, precision, recall, F1, ROC-AUC, RMSE, MAE

3.2 Algorithms

Learn each one by: (a) intuition, (b) math, (c) code from scratch once, (d) scikit-learn version.

Algorithm	Type	Key Idea
Linear Regression	Supervised (regression)	fit a line minimizing squared error
Logistic Regression	Supervised (classification)	sigmoid + probability threshold
Decision Trees	Both	recursive splitting on features
Random Forest	Both	ensemble of trees, bagging
Gradient Boosting (XGBoost/LightGBM)	Both	sequential error-correcting trees
K-Nearest Neighbors	Both	distance-based voting
Support Vector Machines	Classification	maximum-margin hyperplane
K-Means	Unsupervised (clustering)	centroid-based grouping
PCA	Unsupervised (dim. reduction)	project onto max-variance axes
Naive Bayes	Classification	Bayes' theorem + independence assumption

3.3 The ML Workflow

Practice this end-to-end, repeatedly, on different datasets:

- Define the problem & metric
- Collect / load data
- EDA (exploratory data analysis)
- Clean & preprocess (missing values, encoding, scaling)
- Feature engineering
- Train baseline model
- Tune hyperparameters (GridSearch/RandomSearch/Optuna)
- Evaluate on test set
- Interpret results (feature importance, confusion matrix)

Tools

scikit-learn — the standard library for everything above.

Checkpoint projects (pick 3)

- Titanic survival prediction (classification)

- House price prediction (regression)
- Customer segmentation (clustering)

Phase 4: Deep Learning (8–10 weeks)

4.1 Neural Network Fundamentals

- Perceptron -> Multi-layer perceptron
- Forward propagation, backpropagation, chain rule in practice
- Activation functions (ReLU, Sigmoid, Tanh, Softmax)
- Loss functions (MSE, Cross-Entropy)
- Optimizers (SGD, Momentum, Adam)
- Regularization: dropout, batch normalization, L1/L2, early stopping

4.2 Frameworks

PyTorch (recommended — more common in research and most job postings now) OR **TensorFlow/Keras**.
Learn one deeply rather than both shallowly.

4.3 Architectures by Domain

Domain	Architecture	Notes
Images	CNNs (ResNet, EfficientNet)	convolution + pooling
Sequences/Text	RNN -> LSTM -> Transformers	Transformers now dominate
Text (modern)	BERT, GPT-style models	attention mechanism, fine-tuning
Structured/tabular	Usually still gradient boosting	know when NOT to use DL
Generative	GANs, Diffusion Models, Autoencoders	image/audio generation

4.4 Must-Understand Concepts

- Attention mechanism & Transformers (the basis of all modern LLMs)
- Transfer learning & fine-tuning pretrained models
- Embeddings (word2vec, sentence embeddings)

Checkpoint projects (pick 2–3)

- Image classifier with CNN (transfer learning on ResNet)
- Sentiment analysis with a fine-tuned BERT model
- Simple chatbot or text generator

Phase 5: Specialize — Choose 1–2 Paths (6–8 weeks each)

Path A: MLOps / Production ML

- Model deployment (Flask/FastAPI + Docker)
- Model serving (TorchServe, ONNX, TensorFlow Serving)
- CI/CD for ML, model versioning (MLflow, DVC)
- Monitoring drift & retraining pipelines
- Cloud platforms: AWS SageMaker / GCP Vertex AI / Azure ML

Path B: NLP / LLMs

- Tokenization, embeddings, attention deep-dive
- Fine-tuning with Hugging Face Transformers
- Prompt engineering, RAG (retrieval-augmented generation)
- Vector databases (Pinecone, FAISS, Chroma)
- LangChain / LlamaIndex for building LLM apps

Path C: Computer Vision

- Object detection (YOLO, Faster R-CNN)
- Image segmentation (U-Net, Mask R-CNN)
- OpenCV for classical CV preprocessing

Path D: Research / Applied Data Science

- Read papers (start with "Attention Is All You Need", ResNet, GPT papers)
- Reimplement 2–3 papers from scratch
- Statistics-heavy: A/B testing, causal inference, experiment design

Phase 6: Portfolio & Job Prep (ongoing, overlaps Phase 5)

6.1 Build a Portfolio

Aim for 4–6 strong projects, not 20 shallow ones.

- 1 end-to-end project with deployment (a working demo beats a notebook)
- 1 project solving a real problem you personally care about
- Push all code to GitHub with clean READMEs
- Write 2–3 blog posts explaining what you built and why (huge signal to employers)

6.2 Compete & Practice

- Kaggle competitions (start with "Getting Started" comps, then real ones)
- Contribute to an open-source ML repo

6.3 Interview Prep

- Data structures & algorithms (LeetCode — still asked at most companies)
- ML system design ("design a recommendation system", "design a fraud detector")
- Statistics/ML theory questions — be ready to explain bias-variance, regularization, etc. out loud

- SQL (very commonly tested, often overlooked)

Suggested Timeline (part-time, ~12–15 hrs/week)

Month	Focus
1	Python + math foundations
2	Math continued + intro ML concepts
3–4	Core ML algorithms + scikit-learn projects
5–7	Deep learning + PyTorch + projects
8–9	Specialization path
10–12	Portfolio polish + interview prep + applying

Core Resource List

Courses

- Andrew Ng's *Machine Learning Specialization* (Coursera) — best starting point
- Andrew Ng / DeepLearning.AI *Deep Learning Specialization*
- fast.ai *Practical Deep Learning for Coders* — very hands-on
- CS231n (Stanford, free online) — computer vision deep dive
- CS224n (Stanford, free online) — NLP deep dive

Books

- *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* — Aurélien Géron (the single best all-rounder book)
- *An Introduction to Statistical Learning* (free PDF) — theory without heavy math
- *Deep Learning* — Goodfellow, Bengio, Courville (reference, not a first read)

Practice

- Kaggle (datasets + competitions + free notebooks)
- Hugging Face (models, datasets, free course)
- Papers With Code (papers + implementations)

Mistakes to Avoid

- Jumping straight to deep learning without understanding classical ML first
- Only doing tutorials, never building anything from a blank notebook yourself
- Skipping math "for now" and never coming back to it
- Collecting certificates instead of building projects — employers weight projects far higher
- Trying to learn every framework/tool instead of going deep on one stack first

This roadmap is intentionally sequential — resist the urge to skip to deep learning or LLMs before Phase 3 is solid. The fundamentals compound.